

Lab-1 DDPM

112550011 李佑軒

September 22, 2026

1 Introduction

Denoising Diffusion Probabilistic Models (DDPM) generate data by learning to reverse a gradual noising procedure: a forward process slowly corrupts a data sample into pure Gaussian noise, and a neural network is trained to undo that corruption step by step, so that new samples can be produced by denoising from random noise.

In this lab, I implement DDPM from scratch and study it at two different scales. Task 1 models a 2D Swiss Roll distribution, where the sampling process can be plotted directly and thus serves as a sanity check for the implementation. Task 2 scales the same formulation up to image generation, and further explores how different beta schedulers and prediction targets affect the denoising process and the quality of the generated samples.

2 Task 1 - Swiss Roll

In this task, I model a 2D Swiss Roll distribution with a DDPM. I implement **SimpleNet**, a time-conditioned network that predicts the noise added to a sample, together with the core routines of the algorithm: **q_sample** for the forward process, **p_sample** and **p_sample_loop** for the reverse process, and the training loss. Since the data is only two dimensional, both the forward corruption and the learned distribution can be visualized directly.

2.1 Implementation details

2.1.1 TODO #1 - SimpleNet

Code 1: **SimpleNet**: constructor and forward pass

```
1 dims = [dim_in] + list(dim_hids)
2 self.layers = nn.ModuleList(
3     [
4         TimeLinear(dims[i], dims[i + 1], num_timesteps)
5         for i in range(len(dims) - 1)
6     ]
7 )
8 self.out_layer = nn.Linear(dims[-1], dim_out)
9
10 def forward(self, x, t):
11     for layer in self.layers:
12         x = F.relu(layer(x, t))
13     x = self.out_layer(x)
14     return x
```

I build the hidden stack by chaining **dim_in** with **dim_hids**, so that consecutive entries of **dims** give the input/output width of each layer. Every hidden layer is a **TimeLinear**, which applies a linear map and then rescales the result channel-wise by the embedding of t :

$$\text{TimeLinear}(x, t) = \alpha(t) \odot (Wx + b). \quad (1)$$

This is what makes the network *time-conditioned*: one single set of weights is shared across all T timesteps, and the timestep only modulates the activations. Without this, we would need a separate network per timestep, which is clearly infeasible.

The output head is a plain **nn.Linear** with no activation afterwards. The quantity being predicted is the Gaussian noise ϵ , which is unbounded and can be negative, so applying ReLU (or any squashing function) at the output would make it impossible to represent roughly half of the target values. ReLU is only used *between* hidden layers to provide non-linearity.

2.1.2 TODO #2 - q_sample

Code 2: **Forward process** $q(\mathbf{x}_t | \mathbf{x}_0)$

```
1 alphas_prod_t = extract(self.var_scheduler.alphas_cumprod, t, x0)
2 xt = alphas_prod_t.sqrt() * x0 + (1 - alphas_prod_t).sqrt() * noise
```

The forward process is a Markov chain that adds Gaussian noise step by step, but because every step is Gaussian, the marginal at an arbitrary t has the closed form

$$q(\mathbf{x}_t | \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_t; \sqrt{\bar{\alpha}_t} \mathbf{x}_0, (1 - \bar{\alpha}_t) \mathbf{I}), \quad \bar{\alpha}_t = \prod_{s=1}^t (1 - \beta_s). \quad (2)$$

Using the reparameterization trick, sampling from this distribution is written as

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, \quad \epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I}), \quad (3)$$

which is exactly the code above. The key benefit is that we can jump to any timestep in $\mathcal{O}(1)$ instead of simulating t steps of the chain, which is what makes it possible to train on a randomly sampled t in each iteration. The helper `extract` gathers $\bar{\alpha}_t$ for each element of the batch and reshapes it to $(B, 1)$ so that it broadcasts against $\mathbf{x}_0$.

2.1.3 TODO #3 - p_sample

Code 3: **One reverse step** $\mathbf{x}_t \rightarrow \mathbf{x}_{t-1}$

```
1 # 1. predict noise
2 eps_theta = self.network(xt, t)
3
4 # 2. Posterior mean
5 mean = (xt - eps_factor * eps_theta) / alpha_t.sqrt()
6
7 # 3. Posterior variance
8 var = (1 - alpha_bar_t_prev) / (1 - alpha_bar_t) * beta_t
9
10 # 4. Reverse step
11 noise = torch.randn_like(xt)
12 nonzero_mask = (t > 0).float().reshape(-1, *[1] * (xt.ndim - 1))
13 x_t_prev = mean + nonzero_mask * var.sqrt() * noise
```

The reverse step samples from $p_\theta(\mathbf{x}_{t-1} | \mathbf{x}_t) = \mathcal{N}(\boldsymbol{\mu}_\theta, \sigma_t^2 \mathbf{I})$.

Mean. Rearranging Equation 3 gives an estimate of $\mathbf{x}_0$ from the predicted noise, and substituting it into the true posterior mean collapses everything into

$$\boldsymbol{\mu}_\theta(\mathbf{x}_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_\theta(\mathbf{x}_t, t) \right). \quad (4)$$

The template already provides $\text{eps_factor} = (1 - \alpha_t) / \sqrt{1 - \bar{\alpha}_t}$, so the implementation is a direct transcription of this formula. Intuitively, the network estimates how much noise is in $\mathbf{x}_t$, we subtract a scaled version of it, and then rescale by $1/\sqrt{\alpha_t}$ to undo the shrinkage applied by the forward step.

Variance. I use the true posterior variance

$$\sigma_t^2 = \tilde{\beta}_t = \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t} \beta_t. \quad (5)$$

The DDPM paper shows that both β_t and $\tilde{\beta}_t$ work in practice; $\tilde{\beta}_t$ corresponds to the optimal choice when $\mathbf{x}_0$ is drawn from the data distribution. Note that `t_prev` is clamped to 0 so that the index stays valid at $t = 0$; the value it produces there is never used, because of the mask described next.

Adding noise back. Following Algorithm 2 of the paper, the extra noise term is added at every step *except* the last one. At $t = 0$ we want the final clean output, so returning the mean directly avoids re-corrupting the sample we just finished denoising. Instead of branching with an `if`, I use `nonzero_mask`, which is computed per batch element and reshaped to broadcast over the trailing dimensions — this keeps the code correct even when t differs across the batch, and works unchanged for both 2D points and images.

2.1.4 TODO #4 - p_sample_loop

Code 4: Full reverse process

```
1 xt = torch.randn(shape).to(self.device)
2 for t in self.var_scheduler.timesteps:
3     xt = self.p_sample(xt, t.to(self.device))
4 x0_pred = xt
```

Generation starts from pure Gaussian noise $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ and repeatedly applies the single-step denoiser. Since `var_scheduler.timesteps` is constructed as `arange(0, T)[::-1]`, iterating over it already walks $T - 1 \rightarrow 0$ in the correct descending order, so no extra reversal is needed. The `.to(self.device)` call is required because `timesteps` is stored as a plain CPU tensor rather than a registered buffer. After the loop finishes, $\mathbf{x}_0$ is the generated sample.

2.1.5 TODO #5 - compute_loss

Code 5: Noise matching loss

```
1 eps = torch.randn_like(x0)
2 xt = self.q_sample(x0, t, noise=eps)
3 eps_pred = self.network(xt, t)
4 loss = F.mse_loss(eps_pred, eps)
```

This implements the simplified objective (Equation 14 of the DDPM paper):

$$\mathcal{L}_{\text{simple}} = \mathbb{E}_{\mathbf{x}_0, \epsilon, t} \left[\|\epsilon - \epsilon_{\theta}(\mathbf{x}_t, t)\|^2 \right]. \quad (6)$$

The expectation over t is estimated by uniformly sampling one timestep per element in the batch, which is far cheaper than summing all T terms and is unbiased in expectation.

The important detail is that I sample `eps` explicitly and pass it into `q_sample`, instead of letting `q_sample` draw its own noise internally. The loss target must be exactly the same noise that was used to corrupt $\mathbf{x}_0$; if two independent noises were drawn, the network would be regressing towards an unrelated sample and the training signal would vanish.

2.2 q_sample implementation

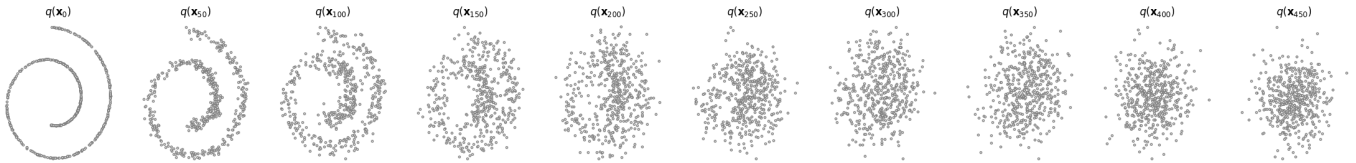


Figure 1: Forward process $q(\mathbf{x}_t | \mathbf{x}_0)$ visualized every 50 timesteps. The Swiss Roll structure is progressively destroyed until the samples are indistinguishable from an isotropic Gaussian.

Figure 1 confirms that `q_sample` behaves as expected. At $t = 0$ the samples trace the clean spiral; by $t = 50$ the curve has thickened but both arms are still clearly separable; around $t \approx 150$ – 200 the spiral has broken up into a diffuse ring; and from $t \approx 300$ onwards the points form a centred isotropic blob with no visible structure left.

2.3 Loss curve and output distribution

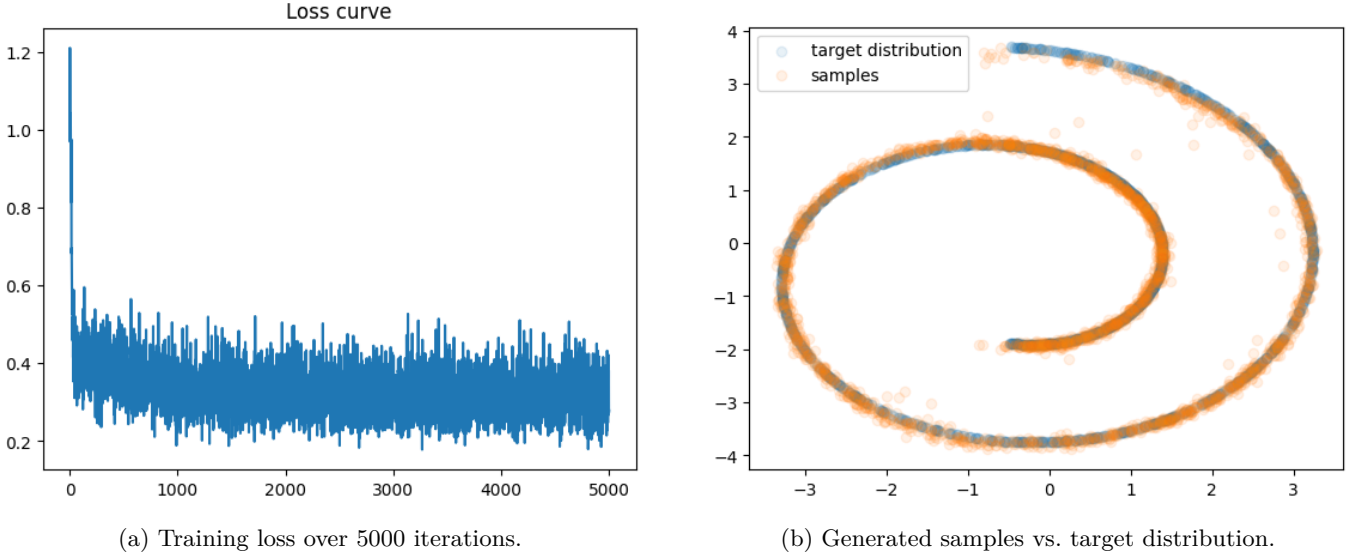


Figure 2: Training curve and sampling result of the 2D Swiss Roll DDPM.

Loss curve. The loss drops sharply from about 1.2 to roughly 0.4 within the first few hundred iterations, then oscillates around 0.3 for the rest of training. The wide band of fluctuation is expected rather than a sign of underfitting: each iteration samples a *different* random t , and at large t the input is almost pure noise, so ϵ is largely unpredictable from $\mathbf{x}_t$ alone. This irreducible component puts a floor on the loss and keeps the per-iteration variance high however well the model is trained.

Output distribution. The model recovers the spiral faithfully: both the outer sweep and the tightly wound inner arm are covered, the two arms stay separated where they run close together, and there is no mode collapse. The main defect is a thin scatter of points lying slightly off the manifold, mostly along the outer arm and near the inner tip. This matches the loss analysis above — the residual error at large t means the early reverse steps place points only approximately.

3 Task 2 - Image Generation

This task applies the same diffusion formulation to a much higher-dimensional distribution: AFHQ animal-face images at a resolution of 64×64 , using a UNet as the prediction network. Besides the components of Task 1, I additionally implement three beta schedulers (linear, quadratic, cosine) and three prediction targets (noise, $\mathbf{x}_0$, mean). I then train the model under different scheduler and predictor combinations, observe the denoising trajectories, and evaluate the sample quality with FID.

3.1 Implementation details

3.1.1 TODO #1 - add_noise

Code 6: Forward process in image space

```
1 alpha_bar_t = extract(self.alphas_cumprod, t, x_0)
2 x_t = alpha_bar_t.sqrt() * x_0 + (1 - alpha_bar_t).sqrt() * eps
```

This is the same closed-form forward process as `q_sample` in Task 1 (Equation 3); only the tensor shape changes from $(B, 2)$ to (B, C, H, W) . Since `extract` reshapes $\bar{\alpha}_t$ to $(B, 1, 1, 1)$, the identical expression broadcasts correctly over images without any modification.

3.1.2 TODO #2 - Cosine beta scheduler

Code 7: Cosine beta schedule (Nichol & Dhariwal, 2021)

```

1 s = 0.008
2 ts = torch.linspace(0, num_train_timesteps, num_train_timesteps + 1)
3 ts = ts / num_train_timesteps
4 alphas_cumprod = torch.cos((ts + s) / (1 + s) * np.pi * 0.5) ** 2
5 alphas_cumprod = alphas_cumprod / alphas_cumprod[0].clone()
6 betas = 1 - alphas_cumprod[1:] / alphas_cumprod[:-1]
7 betas = betas.clamp(max=0.999)

```

Unlike the linear and quadratic modes, which define β_t directly, the cosine schedule defines the *cumulative* product first and then derives β_t from it:

$$\bar{\alpha}_t = \frac{f(t)}{f(0)}, \quad f(t) = \cos^2\left(\frac{t/T + s}{1 + s} \cdot \frac{\pi}{2}\right), \quad \beta_t = 1 - \frac{\bar{\alpha}_t}{\bar{\alpha}_{t-1}}. \quad (7)$$

A few implementation details are worth noting. First, I build $T + 1$ points of $\bar{\alpha}$ rather than T : since each β_t is a ratio of two *consecutive* entries, taking `alphas_cumprod[1:] / alphas_cumprod[:-1]` on $T + 1$ values produces exactly T betas. Second, dividing by $f(0)$ normalizes the schedule so that $\bar{\alpha}_0 = 1$, i.e. no noise has been added at the very beginning. Third, the offset $s = 0.008$ shifts the curve slightly away from $t = 0$, which prevents β_t from becoming vanishingly small at the start — with $s = 0$ the first few steps would add almost no noise and be wasted. Finally, as $t \rightarrow T$ the cosine approaches zero, so the ratio $\bar{\alpha}_t / \bar{\alpha}_{t-1}$ becomes singular and β_t blows up; clipping at 0.999 keeps $\alpha_t = 1 - \beta_t$ strictly positive so that $\sqrt{\alpha_t}$ in the reverse step never degenerates.

The motivation is that the linear schedule destroys information too aggressively near the end of the forward process: $\bar{\alpha}_t$ drops to nearly zero well before $t = T$, so the final steps operate on what is already pure noise and contribute little. The cosine schedule makes $\bar{\alpha}_t$ decay closer to linearly, which spreads the useful signal over a larger portion of the trajectory.

3.1.3 TODO #3 - Predictors

All three predictors sample from the same posterior $q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0) = \mathcal{N}(\tilde{\boldsymbol{\mu}}_t, \tilde{\beta}_t \mathbf{I})$, where

$$\tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0) = \frac{\sqrt{\bar{\alpha}_{t-1}} \beta_t}{1 - \bar{\alpha}_t} \mathbf{x}_0 + \frac{\sqrt{\alpha_t} (1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} \mathbf{x}_t, \quad \tilde{\beta}_t = \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t} \beta_t. \quad (8)$$

They differ only in *how* the network output is turned into the $\mathbf{x}_0$ (or mean) that this formula needs.

Code 8: Noise predictor (`step_predict_noise`)

```

1 is_first = (t == 0).reshape(-1, *(([1] * (x_t.ndim - 1))))
2 alpha_bar_t_prev = extract(self.alphas_cumprod, (t - 1).clamp(min=0), x_t)
3 alpha_bar_t_prev = torch.where(
4     is_first, torch.ones_like(alpha_bar_t_prev), alpha_bar_t_prev
5 )
6
7 # Invert the forward process to recover the predicted clean sample.
8 x0_pred = (x_t - (1 - alpha_bar_t).sqrt() * eps_theta) / alpha_bar_t.sqrt()
9 x0_pred = x0_pred.clamp(-1, 1)
10
11 mean = (
12     alpha_bar_t_prev.sqrt() * beta_t / (1 - alpha_bar_t) * x0_pred
13     + alpha_t.sqrt() * (1 - alpha_bar_t_prev) / (1 - alpha_bar_t) * x_t
14 )
15 var = (1 - alpha_bar_t_prev) / (1 - alpha_bar_t) * beta_t
16
17 noise = torch.randn_like(x_t)
18 nonzero_mask = (~is_first).to(x_t.dtype)
19 sample_prev = mean + nonzero_mask * var.sqrt() * noise

```

Handling $\bar{\alpha}_{t-1}$ at $t = 0$. Indexing with `(t-1).clamp(min=0)` alone would return $\bar{\alpha}_0$, which is wrong; the correct convention is $\bar{\alpha}_{-1} = 1$, so I overwrite that entry with `torch.where`. This is not merely cosmetic: substituting $\bar{\alpha}_{-1} = 1$ into Equation 8 together with $1 - \bar{\alpha}_0 = \beta_0$ makes the second term vanish and reduces the mean to exactly $\hat{\mathbf{x}}_0$, while $\tilde{\beta}_0$ becomes 0. In other words, the final reverse step returns the predicted clean image directly, and the variance term switches itself off. The `nonzero_mask` is therefore redundant in exact arithmetic, but I keep it explicit so that the intent (Algorithm 2 adds no noise on the last step) is visible and no $\sqrt{0}$ gradient issues can arise.

Clamping. $\hat{\mathbf{x}}_0$ is recovered by inverting the forward process, which divides by $\sqrt{\alpha_t}$. At large t this factor is tiny, so any error in the predicted noise is strongly amplified and can push $\hat{\mathbf{x}}_0$ far outside the valid range. Since the training images are normalized to $[-1, 1]$, clamping is a free and effective way to stop that error from propagating into the next step.

Code 9: $\mathbf{x}_0$ and mean predictors (differing parts only)

```

1 # step_predict_x0: the network already outputs x0, so nothing to invert.
2 x0_pred = x0_pred.clamp(-1, 1)
3 mean = (
4     alpha_bar_t_prev.sqrt() * beta_t / (1 - alpha_bar_t) * x0_pred
5     + alpha_t.sqrt() * (1 - alpha_bar_t_prev) / (1 - alpha_bar_t) * x_t
6 )
7
8 # step_predict_mean: the network output is the posterior mean.
9 mean = mean_theta

```

For the **$\mathbf{x}_0$ predictor**, the network directly outputs $\hat{\mathbf{x}}_0$, so the inversion step disappears and the same posterior formula is reused unchanged. Clamping still applies, because the output is meant to be an image.

For the **mean predictor**, the network output is already $\tilde{\boldsymbol{\mu}}_t$, so only the variance term remains to be computed. Importantly I do *not* clamp here: the posterior mean is a blend of $\mathbf{x}_0$ and the noisy $\mathbf{x}_t$, and $\mathbf{x}_t$ is not bounded to $[-1, 1]$, so clamping would systematically bias the trajectory.

3.1.4 TODO #4 - Loss functions

Code 10: $\mathbf{x}_0$ loss and mean loss

```

1 # get_loss_x0
2 t = self.var_scheduler.uniform_sample_t(B, x0.device)
3 x_t, eps = self.var_scheduler.add_noise(x0, t, eps=noise)
4 x0_pred = self.network(x_t, t)
5 loss = F.mse_loss(x0_pred, x0)
6
7 # get_loss_mean: regress onto the TRUE posterior mean, built from the real x0.
8 mean_pred = self.network(x_t, t)
9 mean_true = (
10     alpha_bar_t_prev.sqrt() * beta_t / (1 - alpha_bar_t) * x0
11     + alpha_t.sqrt() * (1 - alpha_bar_t_prev) / (1 - alpha_bar_t) * x_t
12 )
13 loss = F.mse_loss(mean_pred, mean_true)

```

All three losses share the same skeleton: sample a random t , corrupt $\mathbf{x}_0$ with **add_noise**, run the network, and take an MSE against the corresponding target. Only the regression target differs — $\boldsymbol{\epsilon}$, $\mathbf{x}_0$, or $\tilde{\boldsymbol{\mu}}_t$ respectively.

The mean loss is the only one that needs extra work, because its target is not something we already have on hand: it must be computed from Equation 8 using the *ground-truth* $\mathbf{x}_0$ rather than any prediction, which makes it the exact posterior mean instead of an estimate. I reuse the same $\bar{\alpha}_{-1} = 1$ convention as in the sampler so that training and sampling agree at $t = 0$; a mismatch there would mean the network is trained against a target that the reverse process never actually uses.

3.2 Trajectory figures and three beta schedulers

3.3 Result of different Predictors and discuss

3.4 Training and FID score